

Executable Behavioral Design for Industrial Designers

Signature-First Relationships, Progressive Refinement, and a Mechanically Derived Kernel

ZHU ZHEHAO
Research Design Draft

ABSTRACT

Industrial products increasingly embed sensing, temporal logic, software, and control, yet industrial designers still lack a behavioral design medium with the immediacy that CAD provides for geometry. Existing prototyping and visual-programming tools reduce implementation cost but preserve engineering-oriented representations such as state transitions, execution steps, callbacks, or node-level computation. This paper proposes BDL, a Behavior Design Language whose primary artifacts are typed product relationships. A Mapping Block is created from a semantic signature, for example $?f : \text{Tilt} \rightarrow \text{Brightness}$, and may remain unresolved while the surrounding design is authored and checked; the claim is not that designers think signature-first but that an unresolved relationship is a legal, statically meaningful state, so that stopping between declaration and realization costs nothing. The kernel beneath the surface was derived by a mechanized design-space exploration in Lean 4: each construct of an earlier draft was formalized, attacked with counterexamples, and kept only when the tested alternatives failed for a stated reason. What survived is an environment of named declarations with frozen types, monotone public commitments, and write-once realizations, under which refining one declaration preserves every client; nominal semantic types with a representation binding that licenses construction only inside a declaration whose signature announces the concept; physical dimensions carried in primitive operator types; one temporal primitive that reads a clock domain at its previous activation, of which single-domain delay is a special case, with deterministic evaluation that is total exactly on causal designs; nominal clock domains checked by a judgment rather than a type; and nominal physical outputs with one explicit driver each. Signal and event types, effect rows, action requests, and runtime arbitration were removed. A separate validation layer decides, with a solver proved sound and complete for its finite fragment, whether a design fits a declared microcontroller, and its evidence is kept apart from the evidence that survives refinement. The paper states each result with its claim strength, and connects the two through a designer interaction model: a running lamp design carried from named product concepts, through undefined relationships, local formulas, temporal qualifiers, and contexts, to one final target per output, explicit crossings between clock domains, and a board chosen last, with the workspace keeping a valid design distinct from a deployable one. An evaluation plan follows; no elaborator, editor, or user study has yet been built.

CCS CONCEPTS Human-centered computing $\rightarrow$ User interface programming; Software and its engineering $\rightarrow$ Domain specific languages; Theory of computation $\rightarrow$ Type theory; Software and its engineering $\rightarrow$ Formal language definitions; Applied computing $\rightarrow$ Computer-aided design.

KEYWORDS industrial design, product behavior, executable design intent, type systems, synchronous reactive semantics, clock domains, refinement, mechanized design-space exploration, hardware feasibility, design tools

1 Introduction

Industrial design is increasingly concerned with products whose behavior is determined not only by geometry, materials, and mechanisms, but also by sensing, logic, timing, software, and networked control. A cup may infer that it has been lifted, a lamp may adapt to ambient conditions, a medical device may gate an action on multiple safety conditions, and a consumer robot may continuously map sensor estimates to actuator behavior. In such products, behavior is no longer a late implementation detail. It is part of the product concept.

Yet the dominant design media remain asymmetrical. Industrial designers have mature media for shape, layout, appearance, and physical assembly, while product behavior is usually externalized through prose, storyboards, flowcharts, state diagrams, interactive mock-ups, or ad hoc embedded code. The first four are easy to sketch but weak as executable specifications; the last is executable but forces the designer to adopt implementation-oriented concepts such as mutable variables, callbacks, polling loops, and device APIs. Existing physical prototyping systems such as Phidgets and d.tools substantially lowered the cost of building interactive prototypes [1], [2], but their behavioral representations still inherit important assumptions from programming and state-machine formalisms.

A concrete instance of the resulting cost appeared in a two-day introductory hardware workshop that one of the authors designed and taught for ten participants with no prior embedded experience. Two of the available sensors behaved in opposite ways: the ambient-light sensor reports larger values as illumination increases, while the distance sensor reports smaller values as the target recedes. Parti-

pants lost track of which was which, repeatedly and across the whole group. This pattern suggests a representational problem rather than merely a syntactic one. Whether a reading rises or falls with the quantity it measures is a stable fact about a device, and in the code they were writing there was nowhere to record it. It survived only as a sign buried inside an expression and had to be reconstructed each time it was needed. The obstacle was not syntax. A piece of semantic information had no place to live.

This paper proposes a different boundary. The goal is not to make engineering representations merely easier for designers to use. The goal is to define a *native representation of product behavior for design itself*, while retaining enough formal structure for static checking, simulation, and eventual implementation.

We call the proposed system **BDL**, a Behavior Design Language. BDL is organized around a working hypothesis: when a behavioral relationship is first specified, *what kind of relationship should exist* is frequently settled before its exact implementation is. A designer may know that **Tilt influences Brightness** before deciding the transfer function; that **CupPickedUp** should activate a behavior before deciding which sensor and threshold detect pickup; or that a safety condition should suppress an actuator before choosing the device driver. Therefore the primary design artifact should be the *typed relationship*, not the procedure that computes it.

The central example is a Mapping Block. Instead of decomposing a design into procedural steps such as “read tilt,” “calculate brightness,” and “set the LED,” BDL represents one mapping:

$?f : \text{Tilt} \rightarrow \text{Brightness} .$

The mapping may exist before its body. A formula, curve, examples, or a fitted function can later be attached as a definition of the same block. Once a formula is supplied, for example

$$f(\theta) = \text{clamp} \left(0.2 + 0.8 \frac{\theta}{60^\circ}, 0, 1 \right),$$

it inhabits the previously declared signature. The claim attached to this **signature-first** model is deliberately narrow. It is not that designers universally think in signatures before bodies, nor that they should be trained to. It is that an unresolved typed relationship is a legal, statically meaningful state of the design, so that stopping between declaring a relationship and realizing it costs nothing. Whether designers make use of that position, and at what level of task complexity, is an empirical question that this paper leaves open.

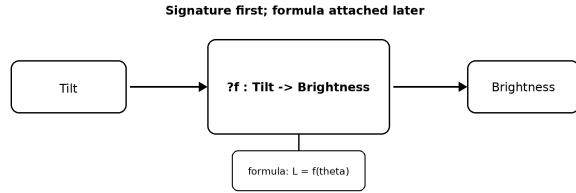


Figure 1: A signature-first Mapping Block. The flow graph contains one semantic relationship, $\text{Tilt} \rightarrow \text{Brightness}$; the formula is attached to the block rather than represented as an additional execution step.

BDL separates a designer-facing surface language from a small formal kernel. The kernel presented here was not designed on paper and then implemented. It was derived by a mechanized design-space exploration in the Lean 4 proof assistant [3]: each candidate construct from an earlier draft of the language was formalized, attacked with counterexamples, reduced to other constructs where possible, and kept only when the tested alternatives failed for a stated reason. The result is smaller than the draft it replaced. Clock-indexed signal types, a separate event type, effect rows, action requests, and runtime actuator arbitration were all removed, each for a reason recorded in the accompanying development. What remains is an environment of named declarations with frozen expected types, monotone public commitments, and write-once realizations; nominal semantic types whose values can be constructed only inside a declaration whose own signature announces the concept; physical dimensions carried in the types of primitive operators; one temporal primitive that reads a clock domain at its previous activation; nominal clock domains with a separate domain judgment; and nominal physical outputs with a single explicit driver each. A validation layer outside the kernel decides whether a design can be placed on a declared target board.

The paper is organized as three views of the same language. The first is the design problem and the vocabulary a designer sees. The second is the formal architecture that gives that vocabulary a precise meaning, stated for each construct with what was proved, what was rejected by counterexample, and what was merely preferred. The third, which connects the two, is the interaction model: what a designer does in the environment over the life of a design, what can be left undecided, and what the tool says when something is wrong. An elaboration architecture and an evaluation plan follow; no elaborator, editor, firmware generator, or user study has yet been built, and the paper does not report one.

2 The Representation Problem

2.1 The target user is not a programmer with fewer syntax skills

Many low-code and visual programming systems reduce textual syntax while preserving the underlying computational ontology: variables, assignments, loops, callbacks, functions, transitions, and scheduling. For software developers this can be convenient. For industrial designers, however, the dominant difficulty is often not syntax but *semantic translation*. The designer begins with a product statement such as “while the cup is held, brightness follows tilt” and must translate it into implementation machinery.

BDL therefore follows a stronger criterion: a surface primitive should be exposed only when it corresponds to a concept that is independently meaningful in the design task. A mutable accumulator used to count samples is generally not such a concept; “three pickup events within ten minutes” is. A polling loop is generally not; “while the product is held” is. A callback is not; “when the button is pressed” is.

The language should directly expose semantic properties, time-varying quantities, discrete occurrences, typed mappings, conditions, behavioral contexts, temporal relations, physical outputs, and safety constraints. By default it should hide program counters, threads, callbacks, continuations, clock variables, and bus transactions. These may remain inspectable in an expert or debugging view, but they are not the primary design medium.

2.2 A flow graph is a dependency view, not a program counter

BDL uses a flow-like canvas because causal and functional paths are useful visual structures. The arrows do **not** mean “execute the left node and then the right node.” They mean that the target relationship depends on the source relationship or value.

Consider the procedural decomposition:

```

[Cup picked up]
|
[Read tilt]
|
[Compute L = f(theta)]
|
[Set LED brightness]
  
```

This decomposition is rejected as a primary design representation because the “read,” “compute,” and “set” nodes are artifacts of an execution model. The corresponding BDL view is a `Held` context containing a mapping from `Tilt` to `Brightness`. The formula is a property of that mapping, and the realization layer later binds the mapping’s value to a physical light output.

The distinction matters because it changes what the designer edits. In a procedure-centric editor, changing an implementation may require rewriting steps and intermediate state. In BDL, changing a transfer function edits the definition of one relationship while preserving the surrounding product logic. This is not only a matter of presentation. Realizing or refining one declaration leaves every other declaration’s typing untouched, and leaves every commitment that was already discharged in place, provided the evidence for those commitments does not depend on what was still unknown.

2.3 Cognitive budget

The surface language must remain intentionally small. Each additional primitive has a cost in learnability, visibility, consistency, and error-proneness, all familiar concerns in the Cognitive Dimensions tradition [4]. BDL therefore uses a *cognitive budget*: a new visible construct is justified only when it captures a distinct design concept that cannot be expressed cleanly as a property of an existing construct.

The same budget was applied to the kernel, under a stricter test. A kernel construct earns its place only if removing it makes some de-

sign either unrepresentable or ambiguous, and the argument for each is a theorem or a counterexample rather than a preference. Several constructs that the surface exposes as distinct concepts turned out, under that test, to be derived forms. Every temporal modifier reduces to one delay primitive. Every cross-domain policy reduces to one transport primitive plus ordinary data. Every output-selection policy reduces to ordinary computation upstream of a single drive edge.

2.4 Progressive disclosure is a semantic property

A design tool can be visually simple and still force premature decisions. BDL instead treats progressive disclosure as part of the language semantics. A design may contain a declaration whose realization is intentionally absent. Later steps may refine the same declaration with mathematical properties, a body, a clock domain, an output binding, and a target board. Each step is either a *refinement*, which preserves everything previously established, or an *edit*, which is permitted but reopens the validation of dependents. That distinction is made precise in the section on declarations, and it is the organizing principle of the interaction model.

3 BDL from the Designer's Side

This section presents the language as a designer encounters it. Every construct here is a surface form; the architecture section states which forms are kernel primitives and which are derived.

3.1 Semantic properties and signatures

The first-class visual object in BDL is a *semantic property*, not a raw scalar. Examples include Tilt, Brightness, Temperature, CupContact, and MotorAngle. Each semantic property has a representation, and may carry units, ranges, and documentation, but its identity is what the language tracks. Tilt and MotorAngle may share an angular dimension without being interchangeable design concepts, and the kernel keeps them apart by identity rather than by representation.

A Mapping Block is created from a signature

$$m : (A_1, \dots, A_n) \rightarrow B.$$

At creation time the implementation may be absent:

$$?m : (A_1, \dots, A_n) \rightarrow B.$$

This is a valid partial artifact. The editor can already reject wires with incompatible types, propagate semantic types downstream, show documentation for the intended relationship, and record additional properties such as monotonicity.

This design deliberately exploits the information density of a function type. Real $\rightarrow$ Real reveals little. Tilt $\rightarrow$ Brightness already communicates much of the design intent. More refined signatures can add dimensions and representation constraints without forcing those details into the main canvas.

3.2 Mapping Blocks

A Mapping Block has a stable identity, a display name, a signature, an optional definition, and a set of declared properties. A definition may be a mathematical expression, a piecewise curve, a set of input-output examples to be fitted, or a reference to an external component. All definition forms elaborate to the same kernel realization, so different authoring styles do not fragment the semantic model.

Three consequences follow from separating identity from definition.

Identity belongs to the signature. The internal identity is stable across every change to the definition; the display name is mutable, so renaming is a refactoring that updates all reference sites rather than a textual edit. Because a mapping can be named before it can be computed, the rest of the model may refer to it, compose with it, and state properties about it while it is still undefined. Declared

properties such as monotonicity attach to the identity, not to any particular definition, and other declarations may rely on them.

The unresolved state is a state of the same object. Behind a Mapping Block is a declaration whose realization is optional. An unresolved block is a declaration with no realization; it is not a separate kind of thing. The word *hole* is retained in the editor as a designer-facing metaphor for this state, because it communicates intentional openness better than “declaration without realization.” It is not a kernel concept.

Attaching a definition is a refinement; replacing or detaching one is an edit. A block may be given a definition once as a refinement step. Replacing the definition with a different one, or removing it, is permitted, but it is an edit: other declarations may have discharged commitments through the old definition, and they must be rechecked. The editor may keep several candidate definitions with one active as a surface convenience; whether this reduces to the write-once kernel realization is an open item.

Nothing in this record imposes an authoring order. Declaring a signature and attaching a formula may be a single action, exactly as a type signature and an equation are written together in a functional language. What the record guarantees is that stopping between the two costs nothing.

3.3 Time-varying values and occurrences

The designer sees two kinds of temporal thing. A quantity such as tilt or temperature has a value whenever its domain is active. An occurrence such as “button pressed” or “pickup detected” may or may not be present at an activation, optionally with a payload.

The surface may display these differently, because they invite different operations. A quantity is mapped, compared, and held; an occurrence is counted, latched, and used to enter a context. Underneath, the distinction is not one of type. Every declaration is a stream under the tick semantics, and an occurrence is a declaration of optional type. Within a single clock domain that is the whole story. Across domains, where several occurrences may fall between two observations, a buffered transport is required, and it is derived from the same two primitives as everything else temporal.

3.4 Time as a modifier, not a waiting instruction

The surface language avoids wait(300 ms) as a primary construct because wait suggests a suspended sequential thread. Instead, BDL uses temporal modifiers that describe relationships:

- p for 300 ms;
- after e by 2 s;
- while p;
- until e;
- since e;
- once e;
- every 1 s;
- rise p, previous x, count e, hold x e.

The designer expresses a temporal property; the elaborator emits the declaration shape that carries the required state. Each of these is a derived form over one kernel primitive. previous, hold, count, since, once, every, and rise were each elaborated and run on a concrete input trace; the duration-qualified forms after, for, while, and until are compositions of these with comparisons and activation and were not separately executed.

3.5 Behavioral contexts

A StateHandler is not a program-counter location. Its surface meaning is:

Within this product context, these behavioral relationships are active.

A context may be activated by a condition, or entered by one occurrence and left by another. It contains a local flow graph and nested contexts. For example, a `Held` context may be active while the cup is not in contact with the table. Inside it, a `Drinking` sub-context may activate when tilt exceeds a semantic threshold.

Nested contexts form a tree, avoiding the need to flatten every orthogonal concern into a Cartesian-product state machine. BDL still borrows the established value of hierarchical state models [5], but the surface metaphor is a *context containing behavior*, not a transition diagram that the user must manually maintain.

StateHandler remains useful at the surface because it gives a name and a boundary to a product context. In the cases examined here, however, that boundary introduces no new execution mechanism. Activation is a Boolean declaration, entry is its rising edge, local temporal state is a delayed cell gated by activation and reset on entry, an inactive context contributes a default, and choosing between the outputs of two contexts is an ordinary conditional in the one declaration that drives the output. The cases examined are condition-scoped activation, entry, reset-on-entry state, inactive default, event-latched activation with exit-wins, state-local output selection, and nested selection with an output. Contexts that carry their own clock domain, and nesting across independently clocked contexts, were not examined, and nothing here says how they would elaborate.

3.6 Physical outputs

A design computes values; it does not move hardware. Physical effect happens only through an explicit binding of one declaration to one named physical output, such as a light channel or a motor. Where several behaviors would influence the same output — a safety override and an interaction context, say — they are combined by an ordinary declaration that becomes the single driver of that output. The combination rule, whether priority, blend, maximum, or clamp, is written in the design, where it can be read, rather than resolved by a runtime policy.

This replaces the request-and-policy model of an earlier draft of BDL. The change is discussed in the section on physical outputs, where the tested alternatives are shown either to duplicate the single-driver rule or to move the conflict into a collector that must itself be a policy.

3.7 Supplied computation blocks

Some product behavior is genuinely easier to state as code than as a diagram. Recursive filters, Kalman estimators, spectral transforms, and self-tuning controllers are not clarified by being decomposed into wires and formulas. BDL therefore admits computation blocks written in the host language and linked into the generated implementation.

This is the interface across which an engineer supplies a designer with a capability. Because authoring is signature-first, the request can precede the implementation: the designer places `?smooth : Distance -> Distance` and the signature is the specification the engineer works against. The same interface is used by the standard library that ships with the system, so that first-party components cannot rely on facilities denied to third-party ones.

A supplied block is a pure function or a stateful transducer. It may not drive an output. What the kernel can derive for a native mapping must instead be declared for a supplied one: determinism, totality on well-typed inputs, output range, properties relied upon downstream such as monotonicity, worst-case state size, and the update rate it was designed for. These declarations are validation obligations, not typing, and the mechanism by which each is discharged is recorded with it. There is no primitive for reusable stateful components; a compo-

nent used in several places is instantiated into fresh declarations by the elaborator, because temporal state belongs to declarations rather than to functions.

4 Designer Interaction Model

The previous section described what a designer sees. This one describes what a designer does. It is written as the interaction architecture the language implies, not as a report on an implemented tool: no editor exists, and every statement about what a designer would find natural is a hypothesis for the evaluation plan to test. The intention is nonetheless concrete. A reader should be able to picture the workspace, what it contains at each stage of a design, and what it says when something is wrong.

The environment is meant to be closer to a behavioral CAD system than to a visual programming IDE. The main canvas holds named product concepts, the relationships between them, the contexts in which those relationships apply, and the physical outputs the product finally drives. Formulas, units, documentation, declared properties, clock information, deployment bindings, and validation status live in a local inspector attached to whatever is selected. Kernel machinery — concept identities, grants, dependency ranks, domain judgments, solver predicates — does not appear as ordinary vocabulary at all. It can be reached through an explanation view, described at the end of this section, but the designer is not expected to go there.

4.1 A running scenario

Consider a table lamp that responds to being handled. When the lamp is picked up and tilted, its brightness follows the tilt, so that a user can dim it by tipping it. When it is set down, it holds the brightness it last had. The lamp has a small heater in its base to keep a drink warm, and a temperature sensor beside it; above a warning temperature the lamp should pulse, and above a critical temperature the heater must switch off regardless of anything else the lamp is doing. The product is to run on an Arduino Nano, which will be chosen last.

This is small, but it exercises everything the interaction model has to offer: relationships that are known before their formulas, a behavior that depends on a context, history without explicit state, a safety condition that competes with an interaction for the same output, two quantities that update at very different rates, and a deployment step at which the design meets a board.

4.2 Starting from intent

The designer begins by naming the concepts the product is about: `Tilt`, `Brightness`, `Temperature`, `Held`. None of these is a sensor or a number. `Tilt` is the product's orientation as the designer means it, not an accelerometer channel; `Brightness` is how bright the lamp is, not a PWM duty. Creating a concept places it on the canvas and opens an inspector where a description, an expected range, and a unit may be written down — or not.

The first relationship is drawn as an arrow from `Tilt` to `Brightness`. The tool asks for nothing else. The arrow becomes a Mapping Block with the signature `Tilt -> Brightness`, a default name that the designer changes to `dimByTilt`, and an empty body. It is drawn distinctly — the editor may render it as a hole — but not as an error. The design now says that brightness depends on tilt, and says nothing about how.

That statement already does work. A second relationship can be drawn from `dimByTilt` onward; a property such as “increasing in tilt” can be recorded in the inspector; and a wire from `Tilt` directly to a motor-angle concept, had there been one, would be refused. What the designer can do next is unconstrained: attach a formula now, or leave it and continue with the rest of the product. The inspector's status line reads *declared*, and explains that the relationship is named and typed, that other parts of the design may depend on it, and that a definition is still to come.

This is the first answer to why one would use BDL rather than a node editor or a statechart. In a node editor the arrow cannot exist without something to compute; in a statechart the relationship is not a first-class object at all. Here it is the primary object, and its incompleteness is a state the tool understands.

4.3 Refining a relationship locally

When the designer opens `dimByTilt`, the canvas does not change. A local editor appears, offering a formula field, a curve to drag, a table of example pairs to fill in and fit, or a reference to a supplied component. Whichever is chosen, the result is a definition attached to the same block; the surrounding canvas continues to show one arrow from `Tilt` to `Brightness`, not the arithmetic inside it.

Suppose the designer sketches a curve: `dim` at rest, full brightness at about sixty degrees, clamped. The inspector shows the fitted formula alongside the curve, and shows the units it inferred: the input is an angle, the output is a dimensionless level in $[0, 1]$. Because the block's signature already promises `Brightness`, the designer writes a scalar expression and the tool supplies the semantic wrapping; there is no constructor to type. Had the designer instead written an expression that divides `tilt` by a time, the inspector would object in the block's own terms: *this expression produces an angular rate, but this block promises brightness*. The status line moves from *declared* to *defined*, and, once the definition checks, to *type-valid*.

The formula is local detail. It matters that it is not another box on the canvas, because the canvas is meant to remain a diagram of the product's behavior, legible to someone who does not want to read equations. A reviewer sees that brightness follows tilt; a designer who wants the curve opens the block.

4.4 Time and history

The lamp should hold its brightness when set down. The designer selects the wire from `dimByTilt` to the light and adds the qualifier `hold` while not `Held`. Nothing else is needed. There is no previous-value variable to declare, no timer to reset, and no flag to clear; the phrase is the whole of the temporal specification.

The same vocabulary covers the rest of the product's history. "Pulse when warm" becomes a `Warm` condition and an every 1 s phrase on the pulse. "The lamp has been picked up at least once since power-on" is once (`rise Held`). "Picked up more than three times in ten minutes" is count (`rise Held`) within 10 min > 3. Each of these appears as an annotation on a relationship or a condition, not as a node with state inside it.

Two things do surface. Every held or delayed quantity has a value at the first tick, and the tool asks for it — what brightness does the lamp show before it has ever been tilted? — because that value is a product decision, not an implementation default. And the status line, once temporal phrases are present, reports *temporally valid* when every such initial value is given and no relationship depends instantaneously on itself. A design in which the pulse rate depended on the pulse, without any delay between them, would be reported at the loop, in the names of the two relationships, with the suggestion to insert a previous.

That these phrases are all shapes over one delay primitive, and that the primitive reads a clock domain at its previous activation, is the subject of the reactive section. The designer does not need to know it, though the explanation view will show it.

4.5 Contexts as places

The tilt behavior should only apply while the lamp is held. The designer draws a context named `Held`, with its activation condition attached — the lamp is off the table — and drags `dimByTilt` inside it. Visually the block is now within a region; semantically, the relationship applies while the region is active. A `Drinking` sub-context can

be nested inside `Held`, entered when tilt exceeds a threshold and left when it falls back, and given its own behavior: a slower dimming curve, say. There is no state-transition table to maintain and no product of states to enumerate.

What the tool shows about a context is which relationships it contains, what enters and leaves it, and what it contributes to each output while active. Entry and exit conditions are ordinary occurrences, so `rise Held` and the like are available. Contexts with local history reset that history on entry unless the designer marks it persistent; the inspector states which.

The reduction of all this to plain declarations — an activation, an entry edge, gated state, a conditional in the output — is not something the designer manipulates. It is what the explanation view shows and what the reactive section justifies for the cases that were examined. Contexts with their own clocks are outside those cases, and the tool should say so rather than elaborate them silently.

4.6 One output, one final target

The lamp's light is a physical output. The designer connects `dimByTilt` (inside `Held`) to it. Then the warning behavior is built: a `Warm` condition on `Temperature`, and a pulsing brightness while it holds. The natural next action is to connect that pulse to the light as well.

The tool declines. The message is not *single-driver violation*. It reads, on the light output, along the lines of *this output already has a final driver, dimByTilt; combine the two brightness values before connecting the output*. The reason is a product reason: two behaviors that both set the same light have no defined result, and any rule the runtime picked — last wins, highest wins, the one drawn first — would be a decision made silently on the designer's behalf.

The repaired design has one more block:

```
dimByTilt (in Held) ----\
                        -> lampTarget -> light
warmPulse (while Warm) -/
```

`lampTarget` : `Brightness` is an ordinary Mapping Block whose definition states the rule. It might say that the pulse takes priority while `Warm`, or that the two are multiplied, or that the maximum is shown. The critical cutoff is handled the same way: `heaterTarget` takes the heater's demand from wherever it comes and forces it to zero above the critical temperature. The override is visible as a block on the canvas, upstream of the output, rather than as a policy attached to a context. When the designer later asks why the heater is off, the answer is on the canvas.

The status line on an output reads *output-complete* when exactly one final target drives it, and reports each undriven output the product requires until then.

4.7 Values that move at different speeds

The temperature sensor updates once a second. Tilt updates fifty times a second. The designer records this not as two rates but as two domains: `Temperature` moves with the environment, `Tilt` and `Held` move with the interaction. Domains are declared by name on the concept, and at this stage no rate is attached to either.

The first time a relationship reads across the boundary — `heaterTarget`, in the interaction domain, reads the critical-temperature condition from the ambient domain — the tool stops at that wire. Again the message is not a judgment name. It says that the two values update in different domains, and asks how the reader should see the source: as the last value the source produced, with a stated value to use before the source has ever reported, or by moving the reader into the source's domain and accepting a slower response. Both are legitimate designs. Choosing the first adds a visible transport on the wire, and the initial value it asks for — is the heater interlock engaged

or released before the first temperature reading? — is exactly the kind of decision that ought to be written down for a safety condition. The status line reports *clock-consistent* when every crossing has been settled this way.

Rates are attached at deployment. Changing a rate later changes what the product does, and the tool will re-run any timing obligations, but it changes nothing about which relationships are valid. Moving a concept from one domain to another does, and the tool treats that as an edit: it reopens every relationship that read the concept.

4.8 Choosing a board

Until this point the design has said nothing about hardware. The physical outputs are named — `light`, `heater` — and each has a kind: the light is a PWM channel, the heater is a switched load. The sensors are declared by kind too: the tilt estimate comes from an inertial sensor on the I2C bus, the temperature from an analog input.

Selecting an Arduino Nano in the deployment panel derives, from those kinds, what the design needs from a board — one PWM line, one digital output, two bus lines on the same I2C unit, one analog input — and attempts to place them. For the lamp this succeeds immediately, and the panel proposes an allocation: the light on a PWM pin, the heater on a digital pin, the sensor on the two I2C pins. The designer may accept it, or pin a requirement by hand — the light on D3, because that is where the board’s connector is — and let the rest be placed around it. A manual pin is a constraint on deployment, not a change to the design: nothing on the canvas moves.

The shape of a conflict is easier to see on a larger design, and the case examined in the development is the one to picture. A product with four motor channels, each an H-bridge needing a PWM line and a direction line, and an inertial sensor on I2C, fits the Nano; the allocation the solver produced is `M1 -> D3/D0`, `M2 -> D5/D1`, `M3 -> D6/D2`, `M4 -> D9/D4`, `IMU -> A4/A5`. A product needing seven independently dimmed channels does not fit, because the Nano has six PWM pins. The behavioral design is untouched by this; nothing on the canvas turns red. The deployment panel reports, on the seventh channel, that seven PWM lines are required and six are available, and names for each PWM pin the channel occupying it. A designer who then selects a larger board sees the same design placed without change. A designer who pins a PWM channel to D3 and later adds a rotary encoder that needs D3 as an interrupt line is told, on the encoder, that the only remaining interrupt pin is held by a manual assignment.

The distinction the panel is built to keep visible is that a design can be entirely valid — typed, temporally sound, every output driven — and still not fit the chosen board. The two are reported in different places and worded differently, so that neither is mistaken for the other.

4.9 What the workspace says at each stage

Figure 2 lists the states a design passes through. They are not a row of compiler badges. Each is meant to answer three questions about the selected object: what remains unresolved, what is already settled, and what can be done next.

State	What is settled	Where
declared	the relationship is named and typed; others may depend on it; no definition yet	canvas
defined	a formula, curve, examples, or component is attached	inspector
type-valid	the definition produces what the signature promises; declared properties hold every held or delayed value has a first-tick value; no instantaneous loop	inspector
temporally valid	every read across domains has been given a transport and an initial value	canvas
clock-consistent	every physical output the product requires has exactly one final target	canvas
output-complete	the chosen board can carry every derived requirement	outputs
hardware-feasible		deployment

Figure 2: States a design passes through, as the workspace reports them. All but the last are properties of the design alone; the last is a property of the design together with a board, and is re-established whenever either changes.

A relationship that is declared but not defined is not flagged as wrong; it is shown as open, and the design around it is checked as far as it can be. A relationship that is defined but not type-valid is flagged at the definition, in the vocabulary of the block. A loop or a missing initial value is flagged at the wire. A missing final target is flagged at the output. A conflict on the board is flagged in the deployment panel, on the requirement that could not be placed.

Two states deserve their own visibility. *Hardware-feasible* is separate from everything above it, because it is the only state that depends on something other than the design, and it is re-established from scratch when the board changes or the design grows; six channels fit, a seventh may not, and nothing about the first six changes. And a property discharged because a supplied component’s author asserted it is not shown the way a property the tool computed is shown. A latency bound that rests on a declared worst-case execution time is weaker than one derived from the design, and the two must not look alike.

4.10 Looking underneath

Most of the time the compact surface is all a designer sees. When behavior is surprising — the lamp pulses once more than expected, or the heater takes a second longer than expected to cut off — an explanation view opens the selected object to show what it became. For `hold while not Held` it shows the `Held` value as a delayed cell and the condition that gates it. For a context it shows the activation, the entry edge, and the reset. For a cross-domain wire it shows the transport and the tick at which the source was last read, which is where the extra second lives. For an output it shows the final target and, once a board is chosen, the requirements derived from the output’s kind and the pins they were given.

This view is the answer to the hidden-elaboration risk noted in the discussion. The surface stays simple because elaboration does real work; the explanation view exists so that the work is inspectable rather than mysterious. It is also where the kernel vocabulary is allowed to appear, for an engineer who receives the design and wants to know exactly what was generated.

4.11 The workflow as a whole

Read end to end, the intended sequence is this. The designer names the concepts the product is about and draws the relationships between them, leaving each undefined until there is something to say. Relationships are refined locally, by formula, curve, examples, or a supplied component, without the canvas changing shape. Time and history are stated as qualifiers on relationships. Contexts give a name and a boundary to the situations in which behaviors apply, and are nested rather than multiplied. Where two behaviors reach for one output, the tool asks for one final target, and the rule that combines them becomes a visible block. Where two quantities move at different speeds, the tool asks how one should see the other, and

the answer becomes a visible transport with a stated initial value. Only then is a board chosen; the design's needs are derived from the kinds of its inputs and outputs, an allocation is proposed or a conflict is reported, manual pins are honoured, and switching boards re-solves the same needs without touching the design. Throughout, the workspace reports what is settled and what is open, and keeps a design that is valid distinct from a design that is deployable.

Whether designers work this way when given the chance, and whether it helps them, are the questions the evaluation plan is written to answer.

5 From Surface to Kernel: Architecture

The system has three layers, and the boundary between them is the main architectural result of the formal development.

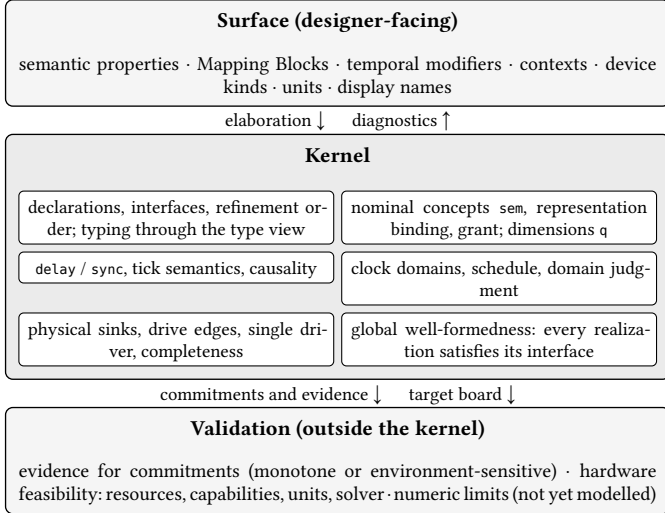


Figure 3: The three layers. Typing consults only the type view of declarations and the representation view of concepts; validation may consult commitments, evidence, and the target; the surface is derived forms over the kernel.

The **surface** is what the designer authors: semantic properties, Mapping Blocks, temporal modifiers, contexts, device bindings, units, and display names. Everything in it elaborates to kernel objects, and the elaboration is one-directional: the kernel never needs to recover surface structure.

The **kernel** is the formal object of this paper. It consists of an environment of declarations, a typing judgment, a tick-indexed evaluation relation over one or several clock domains, a domain judgment, and a small number of global well-formedness conditions: every realization satisfies its interface, the instantaneous dependency graph is acyclic, every reference respects domains, and every physical output has at most one driver. The typing judgment reads only the *type view* of declarations — their expected types — and the *representation view* of concepts. It does not read realizations, commitments, evidence, clocks, or bindings.

The **validation layer** is everything that may depend on more than types. It discharges the commitments a declaration makes, and it decides whether a design fits a target board. Two kinds of evidence live here and are kept apart. Evidence that is meant to survive refinement — a monotonicity commitment discharged compositionally through the commitments of other declarations — must be stable under monotone extension of the environment, and the kernel imposes that condition. Evidence that is not meant to survive refinement — the existence of a pin assignment on a particular board — is re-established after every change and is never merged with the first kind.

Figure 3 shows the layers. What is notable about the arrangement is how much of the earlier draft of BDL is absent from the kernel band. Reactive types, event types, effect rows, action requests, policy transformations, and a five-phase tick with a resolve step were all part of the draft kernel. Each was removed because it either added no rejection the smaller kernel lacked, or made a design decision on the designer's behalf that should have been visible in the design. The states of Figure 2 are the designer-facing face of the same structure: *declared* and *type-valid* are the typing judgment and satisfaction; *temporally valid* is causality; *clock-consistent* is the domain judgment; *output-complete* is the single-driver and completeness conditions; *hardware-feasible* is the validation layer's solver. Each is decidable for finite designs.

6 Declarations, Interfaces, and Refinement

The foundational kernel object is a **design declaration**,

$$\text{DesignDecl} = \langle \text{id} : \text{DeclId}, \\ \text{interface} : \text{DeclInterface}, \\ \text{realization} : \text{Option Expr} \rangle,$$

where an interface is an expected type together with a set of public commitments:

$$\text{DeclInterface} = \langle \text{expectedType} : \text{Ty}, \\ \text{commitments} : \text{PropertyId}^* \rangle.$$

A design is an environment $\Delta : \text{DeclId} \rightarrow \text{Option DesignDecl}$. An unresolved declaration is one whose realization is none; nothing else distinguishes it. Display names are not part of the kernel.

6.1 Typing through the type view

Terms refer to declarations by identity, $\text{declRef } d$. The typing judgment $\Theta; \Delta; G; \Gamma \vdash e : \tau$ takes a concept environment Θ and a grant G , both introduced in the next section, and a declaration environment Δ which it consults through exactly one projection, the type view $\Delta^{\text{ty}}(d)$, the expected type of d if declared:

$$\frac{\Delta^{\text{ty}}(d) = \text{some } \tau}{\Theta; \Delta; G; \Gamma \vdash \text{declRef } d : \tau}.$$

This is the only rule that reads Δ . A client of d is therefore typed against d 's interface and never against its body, whether or not a body exists. Inference is syntax-directed and decidable, and an inference function is proved sound, complete, and unique against the judgment.

6.2 Satisfaction, well-formedness, and refinement

A realization e **satisfies** an interface S when it has the expected type under the grant of that type and discharges every commitment:

$$\text{Satisfies } ev \ \Theta \ \Delta \ \Gamma \ e \ S := \\ \Theta; \Delta; \text{Grant.of}(S.\text{ty}); \Gamma \vdash e : S.\text{ty} \\ \wedge \forall p \in S.\text{commitments}. ev \ \Delta \ e \ p,$$

where $S.\text{ty}$ abbreviates the expected type. Here $ev : \text{DeclEnv} \rightarrow \text{Expr} \rightarrow \text{PropertyId} \rightarrow \text{Prop}$ is an abstract **evidence** relation supplied by the validation layer. It takes the environment as an argument because compositional discharge needs it: “ A is monotone because B is committed to be monotone” consults B 's interface. A design is **globally well formed** when every stored declaration sits under its own identity and its realization, if any, satisfies its interface in that design.

Interfaces are ordered by monotone refinement: $S \sqsubseteq S'$ when the expected type is unchanged and the commitments of S are contained in those of S' . A declaration takes a refinement step in one of three ways. An unresolved declaration may have its interface refined; an

unresolved declaration may be realized with a satisfying body; and a realized declaration may have its interface strengthened, provided the body is re-verified against the new interface. The reflexive-transitive closure of these steps is exactly the structural order together with well-formedness of the target, where the structural order DeclLeq requires the same identity, interface refinement, and a write-once realization, and $\text{EnvRefines } \Delta_1 \Delta_2$ lifts it pointwise while permitting new declarations.

6.3 Client stability

Can a declaration be refined or realized without editing its clients, and without invalidating what was previously established about them? The answer has two halves with deliberately different hypotheses.

The typing half needs only the structural order. If B is declared in Δ and $\text{DeclLeq } B B'$, then every typing judgment in Δ holds in $\Delta[B']$. This is a one-line consequence of typing references through the type view, and it should be read as such: its content is that the decision to let clients see interfaces and never bodies is *sufficient* for client stability. It is also necessary. Change B 's expected type while keeping its identity, and every client breaks.

The commitment half needs more. If Δ is globally well formed, B takes a refinement step whose side conditions are checked in Δ , and the evidence relation is **monotone** — stable under EnvRefines — then $\Delta[B']$ is globally well formed, and the same holds for a whole lifecycle of B checked against the original environment. The monotonicity hypothesis was not part of the original design. It appeared when the theorem was attacked. An evidence relation that consults the *absence* of information — one that discharges a property because a dependency is still unresolved, say — is destroyed by a perfectly valid realization step, and the non-monotonicity of that evidence can be derived from the failure of preservation. Any discharge mechanism intended to survive refinement must therefore be positive in the environment.

6.4 Refinement versus edit

The preservation theorems cover refinement only. The table lists the operations examined and their classification; each row is witnessed by an example on a two-declaration design in which $A : \text{nat} \rightarrow \text{bool}$ is realized through an unresolved $B : \text{nat} \rightarrow \text{nat}$.

Operation on B	Kind	Effect on A
add a public commitment	refinement	typing and commitments preserved
realize	refinement	preserved; A now unfolds to a closed program
strengthen a realized interface	refinement, with re-verification	preserved
change the expected type, keep identity	edit	typing broken
drop a commitment	edit	typing silent; A 's commitment broken
replace B by a new identity	edit	dangling reference
add a new B' beside B	edit	A still depends on the stale B ; never executable
detach or replace the realization	edit	evidence that consulted the body is void
assign or change a clock domain	edit	domain judgment on clients broken
retarget an output binding	edit	completeness or single-driver may break

Two rows deserve comment. Dropping a commitment changes no type, so the type checker is silent, yet A 's own commitment was discharged through B 's and is now unsupported. Commitments are therefore part of the interface in the same load-bearing sense as the expected type. That is a stronger position than the earlier draft of

this paper took when it described properties as merely attaching to a name. Detaching a realization, which that draft permitted as an ordinary operation, is an edit for the same reason: clients' typing is unaffected, but any evidence that consulted the body is void. The kernel does not forbid edits. It declines to promise anything about them, and the tool must reopen the validation of transitive dependencies.

6.5 What persistent identity is

The earlier draft left open whether a persistent, referable identity for an unresolved relationship is a novel abstraction. It is not. Every theorem that mentions identity uses it only to make an update land on the slot a reference resolves to, which is what a name does in any environment-based semantics. The non-trivial content lies in the environment order, and in the two facts that clients depend on it only through the type view for typing and only through monotone evidence for commitments. This is the familiar interface/implementation separation of module signatures, of a parameter later given a definition, or of a metavariable context with write-once assignment and fixed types, together with a Kripke-style stability condition on evidence. What is slightly non-standard is that the interface carries a growable commitment set whose growth is a first-class operation on a declared-but-undefined name, and that the kernel imposes a stability condition on the validation layer. Neither is a new type-theoretic mechanism, and the paper does not claim one.

7 Semantic Identity and Representation

7.1 Nominal concepts

Suppose concepts were represented only by their representation types, so that Tilt and MotorAngle are both numbers. Then the wire $\text{motorTarget} := \text{tiltSensor}$ is well typed and the design is globally well formed, because nothing in the model records the distinction. This baseline was built, the wire was accepted, and three ways of recording the distinction were then tried against it.

The one that survived is a single nominal type constructor over an internal identity:

$$\text{SemanticId}, \quad \text{Ty} \ni \text{sem } s.$$

Two distinct identities are distinct types regardless of representation, so the invalid wire is rejected by the ordinary rules of the simply typed calculus with no additional judgment. An explicit relationship between concepts, $\text{tiltToMotor} : \text{Tilt} \rightarrow \text{MotorAngle}$, is an ordinary declaration of arrow type — a design relationship that is itself signature-first and may remain unresolved — and it appears in the term wherever a crossing occurs. It is not a cast, coercion, or conversion; the kernel has no such mechanism.

Semantic identity is independent of declaration identity, of display name, of dimension, and of hardware, and each independence was tested rather than assumed. A rename preserves identity, whereas a model in which the name *is* the identity makes renaming destructive. Treating a concept as an ordinary declaration admits two category errors: the concept becomes usable as a value, and it can be realized by a number. Keeping identity out of the type as interface metadata, checked by a direct-wire rule, is evaded by η -expansion, since $(\lambda x. x) \text{ tilt}$ has the same flow with no direct wire. A compositional role judgment strong enough to close that gap has the rule shapes of typing over types-with-sem, and the one such formulation examined duplicated nominal typing without benefit. Flow-sensitive or relational semantic analyses were not formalized and are not ruled out; nominal sem is the smallest mechanism among the designs that were tried, not the only one possible.

Erasing all semantic identities is sound — a well-typed semantic term is well typed at its representation — and the baseline is exactly what

erasure leaves. Generated code is thus ordinary code; the semantic layer has no runtime residue.

7.2 Representation binding and the grant

Nominal identity alone leaves semantic values opaque. Without a way to observe a representation and construct a value, no mapping can be realized by a formula; under nominal typing alone a semantic value can only originate from a declaration of semantic type. That is the right state before representation is added. The question is how to add it without destroying what identity just bought.

The obvious form — global $\text{rep}_s : \text{sem } s \rightarrow R$ and $\text{mk}_s : R \rightarrow \text{sem } s$ available to every term — destroys it immediately. $\lambda x. \text{mk}_{\text{Motor}}(\text{rep}_{\text{Tilt}} x)$ is a well-typed $\text{Tilt} \rightarrow \text{MotorAngle}$ in the empty environment, with no declaration and no mapping; a motor angle can be manufactured from a literal; and the crossing can hide inside a body whose signature mentions no motor. Observation alone, with no construction, is safe but cannot realize a mapping. The surviving model separates the two:

- a **concept environment** $\Theta : \text{SemanticId} \rightarrow \text{Option Ty}$ binds each concept, write-once, to a representation that mentions no semantic type and contains no function type;
- $\text{rep } e$ is typed at R whenever $e : \text{sem } s$ and $\Theta s = \text{some } R$, everywhere;
- $\text{mk } s e$ is typed at $\text{sem } s$ whenever $e : R$, $\Theta s = \text{some } R$, and the grant permits s .

$$\frac{\Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : \text{sem } s}{\Theta; \Delta; G; \Gamma \vdash \text{rep } e : R}$$

$$\frac{G s \quad \Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : R}{\Theta; \Delta; G; \Gamma \vdash \text{mk } s e : \text{sem } s}$$

The grant G is a predicate on concepts. Client code is typed under the empty grant. The realization of a declaration is typed under $\text{Grant.of}(\tau)$, the concepts in result position of its own signature τ . A value of MotorAngle can therefore be constructed only inside a declaration that announces MotorAngle in its signature, which is exactly where a reader of the design would look for it. A well-typed term constructs s only where granted s ; the hidden crossing above is rejected under the grant of an unrelated declaration and becomes legal, and visible, once tiltToMotor is declared; binding an unbound concept is monotone for typing, satisfaction, and global well-formedness; and rebinding a concept to a different representation is an edit that breaks existing realizations.

Two constraints on the representation were not anticipated. Representation types must be free of semantic types, because if Tilt may be represented by MotorAngle then rep itself is a hidden mapping under every policy, including observation-only. And they must be data types, a requirement that arrived later from the reactive semantics: a semantic value may be delayed, and a function-typed representation would carry a closure across ticks.

The grant is a known shape — a capability attached to a definition site, or equivalently the private constructor of an abstract type exported only to the module that declares it. Its contribution here is where the capability comes from: the signature the designer already wrote, so no annotation is added. The paper does not present it as a capability calculus. One consequence should be stated plainly. After all bodies are inlined into one executable program, that program is checked under the universal grant, because each construction was authorized at its own declaration. Semantic isolation is a property of the design graph and survives inlining as provenance, not as a type property of the executable.

7.3 Physical dimensions

Physical quantities have type $q d$ for a dimension d , an exponent vector over a small set of base dimensions. There is no dimension-specific typing rule; the algebra lives entirely in the types of registered primitive operators,

$$\begin{aligned} \text{add}_d &: q d \rightarrow q d \rightarrow q d, \\ \text{mul}_{d_1 d_2} &: q d_1 \rightarrow q d_2 \rightarrow q (d_1 + d_2), \\ \text{div}_{d_1 d_2} &: q d_1 \rightarrow q d_2 \rightarrow q (d_1 - d_2), \end{aligned}$$

and an application is checked by ordinary function application. Erasing every dimension to the zero vector is sound and accepts length + time, so the untyped numeric baseline is the erasure of dimensional typing in the same sense that the representation baseline is the erasure of nominal typing. Dimensions might instead have been validation metadata; the argument against is that multiplication and division *produce* dimensions, so any checker recomputes the same inference, but that family was argued against rather than excluded.

Dimension and semantic identity are orthogonal. Tilt and MotorAngle both bound to $q \text{Angle}$ remain distinct types. A mapping realized by the dimensioned formula $\lambda x. \text{mk bright}(\text{rep } x \cdot \text{gain})$ with $\text{gain} : q (\text{Angle} - \text{Angle})$ is typed; a dimension error inside the formula is caught by the same typing; and the formula cannot manufacture a MotorAngle despite the shared dimension. The association between a concept and its dimension lives in Θ , not in the identity and not in the type constructor. The earlier draft's two-index $\text{Sem}[n, d]$ becomes $\text{sem } s$ together with $\Theta s = \text{some } (q d)$.

Units are surface. A literal $n \text{ u}$ elaborates to a dimensioned literal scaled by the unit's factor; changing the unit changes the value, never the type, and mixed-unit addition works after elaboration. Expressing a quantity in a unit — its *coordinate* — and building a quantity from a coordinate are the same arithmetic against a unit constant: $\text{inUnit}(q, u)$ is q divided by the unit's scale and has dimension zero, $\text{withUnit}(x, u)$ is x times the scale and has the unit's dimension, and a conversion between two units is their composition; each is elaborated, none is a kernel construct, and a unit choice never reaches a type (1 m and 100 cm are equal values of one type). The unit laws — round trips, derived conversion, dimension safety — are proved over an abstract scalar domain and instantiated by a symbolic group in which π is a generator, so that a degree is exactly $\pi/180$ radian; the executable kernel truncates to naturals and production approximates in floating point. A preferred display unit is presentation: it changes the number shown and no judgment of the design. Affine scales such as degrees Celsius are not linear (0°C is 273.15 K), yet their literals and coordinates elaborate exactly by adding an offset; what the dimension cannot express is that the sum of two absolute temperatures is not an absolute temperature — a point/difference sort the surface language does not yet have, so Celsius arithmetic remains deferred while Celsius conversion does not.

8 A Minimal Reactive Semantics

8.1 One primitive

The reactive kernel adds one expression form and no types:

$$\text{delay init } e.$$

Its meaning is given by a tick-indexed big-step evaluation relation $\text{Ev } \Delta I t \rho e v$: the value of e at tick t under a local environment ρ , where $I : \text{DeclId} \rightarrow \mathbb{N} \rightarrow \text{Value}$ supplies the inputs. Every declaration is a stream by interpretation. An unresolved declaration is an input and takes its value from I ; a realized declaration is evaluated from its body at the current tick; $\text{delay init } e$ evaluates e at tick t when read at tick $t + 1$, and init at tick 0:

$$\frac{\Delta^{\text{real}}(d) = \text{none}}{\text{Ev } \Delta \ I \ t \ \rho \ (\text{declRef } d) \ (I \ d \ t)}$$

$$\frac{\text{Ev } \Delta \ I \ 0 \ \rho \ \text{init } v}{\text{Ev } \Delta \ I \ 0 \ \rho \ (\text{delay init } e) \ v} \quad \frac{\text{Ev } \Delta \ I \ t \ \rho \ e \ v}{\text{Ev } \Delta \ I \ (t + 1) \ \rho \ (\text{delay init } e) \ v}$$

There is no signal type in Ty. Under this semantics a signal type would be inhabited by exactly the terms of the underlying type and would reject nothing. The information a reactive type would carry in a multi-domain setting is *which clock*, and the next section places that information in a judgment rather than a type. There is likewise no event type. Within one domain an input delivers at most one value per tick by construction, so an occurrence is a stream of optional type, and the streams of type $\text{opt } \tau$ are exactly the streams of multiplicity at most one. What separates an occurrence from an optional value — two occurrences falling in one observation interval — can only be seen when a source ticks faster than its observer. That is a cross-domain question and is treated there.

Two restrictions on delay were forced by the totality proof rather than chosen. The delayed type must be a **data** type, one with no function type inside, because a delayed closure would have to persist across ticks and the logical relation for closures is tick-indexed and cannot be transported. And delay may occur only at **top level**, under no binder, because a delay under a lambda would re-evaluate its operand at the previous tick in an environment created at the current tick. Temporal state therefore belongs to declarations, and mappings are pointwise. This is the arrangement of *pre* in Lustre, where it lives in nodes rather than in functions [6], and its practical consequence is that a reusable stateful component is instantiated into fresh declarations rather than abstracted over.

8.2 Causality

The dependency graph on declarations comes in three variants. Structural dependency, *DependsOn*, records every reference in a body. Instantaneous dependency, *InstDependsOn*, excludes references under the delayed operand of a delay (the initial value is read at tick 0 and counts as instantaneous). A design is **causal** when its instantaneous graph is acyclic, witnessed by a bounded rank:

$$\text{Causal } \Delta := \exists \text{rank}, R. (\forall d. \text{rank } d < R) \wedge \forall a b. \text{InstDependsOn } \Delta \ a \ b \rightarrow \text{rank } b < \text{rank } a.$$

On the delay-free fragment this coincides with structural acyclicity, so the earlier acyclicity condition is the timeless special case rather than a replaced requirement. A structural cycle every one of whose paths passes through a delayed operand — $A := \text{delay } \Theta \ B$; $B := A$, or a self-delayed accumulator — is causal and runs at every tick. A cycle that is partly delayed is not causal. A strict cycle, one through neither a delay nor a lambda, has no value at any tick.

Evaluation is a partial function: one tick, one environment, one term, at most one value, with no hidden evaluation order, and this holds unconditionally. It is total on causal designs: if Δ is causal and globally well formed and the inputs are well typed, every declaration has a value at every tick, related to its type by a logical relation, by an induction on tick, rank, and derivation. An executable interpreter is proved sound for the relation, and every trace reported in this paper was obtained by running it.

One gap should be recorded. A cycle guarded by a lambda, $A := \lambda x. A \ x$, is rejected by Causal, yet *declRef* A does evaluate — to a closure; only applying it diverges. Causal is conservative for lambda-guarded cycles, and the negative theorem covers strict cycles only.

8.3 Derived operators

Every temporal operator of the surface language reduces to delay and the primitive operators. There is no independent kernel defin-

ition of count for the reduction to be proved equivalent to; what was done instead was to elaborate each operator, check its typing and causality, and run it on a concrete input trace. In each row the declaration refers to itself. Each is a self-delayed cycle, the class that structural acyclicity forbade and causality licenses.

Surface	Declaration shape
previous x	delay init x
previous x without init	delay none (some x); the absence is pushed to consumers
hold init e	getD e (delay init self)
count e	ite (isSome e) (1 + delay 0 self) (delay 0 self)
since e	ite (isSome e) 0 (1 + delay 0 self)
once e	delay false self v isSome e
every n	modulo- <i>n</i> counter over delay
rise b	$b \wedge \neg \text{delay false } b$, as an optional Boolean

State has no identity of its own. A cell is a delay in a declaration body, and nothing refers to it because consumers refer to the declaration. There is consequently no notion of two writers to one cell in this kernel.

State preserves semantic identity and dimension. The typing rule is $\text{delay} : \tau \rightarrow \tau \rightarrow \tau$ for data τ , so a delayed tilt is a tilt and a backward difference over a time step has dimension *Length* — *Time* with no derivative primitive. Provenance holds in the reactive setting too: if no signature announces a concept and no input carries it, no value at any tick carries it. Temporal state carries tags; it never creates them.

Initialization is semantic, not validation. Every delay carries an explicit initial value. Two toy relations without one show why: the first tick is either undefined or nondeterministic. Adding or removing a delay, or changing an initial value, is an edit.

9 Clock Domains

9.1 Identity, not rate

“Contact and orientation move with the interaction; temperature moves with the environment.” A designer can say this before any rate is known, and it is a statement about which quantities are updated together, not about how often. BDL records it as a nominal **clock domain**, *ClockId*, and treats rate as validation data that never enters the kernel.

The time model is one global base tick and a schedule $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ saying at which global ticks each domain activates. A period n induces the schedule $t \bmod n = 0$; the schedule lives outside the design. Domain-local time is not a separate counter but the sequence of a domain’s activations.

Each non-agnostic declaration is assigned a domain by a **clock environment** $K : \text{DeclId} \rightarrow \text{Option ClockId}$; a declaration with no domain is a pure mapping that may serve any domain. The clock is interface data in every sense that matters — clients’ validity depends on it, it is frozen under refinement, and changing it is an edit — and it is stored as a projection beside the interface, exactly as a concept’s representation is stored in Θ rather than in the type. Whether to fold it into the interface record is churn rather than semantics.

Rate and identity are distinct. A clone of a domain with the identical schedule is a different domain, and a direct wire between them is rejected; a domain at the same rate but shifted in phase reads different values through a transport. Rate changes are validation-only. They change the induced schedule and hence the observed values, but no client’s well-formedness. This is where BDL departs from synchronous languages that recover clocks by inference [7]. The domain is authored, because the information needed to infer it does not arrive until realization binding, which in this workflow is the point at which the designer is least able to make the decision.

9.2 One transport primitive

Cross-domain reading is the second and last temporal form:

$\text{sync } c \text{ init } e,$

the value of e , evaluated in domain c , at the last activation of c strictly before the current tick, and init if there has been none. The multi-domain evaluation relation $\text{MEv } S \Delta I c t \rho e v$ indexes evaluation by the domain in which it takes place, and its two transport rules are

$$\frac{\text{prevAct } S c' t = \text{none} \quad \text{MEv } S \Delta I c t \rho \text{ init } v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{ init } e) v}$$

$$\frac{\text{prevAct } S c' t = \text{some } t' \quad \text{MEv } S \Delta I c' t' \rho e v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{ init } e) v}.$$

delay is sync at the expression's own domain: $\text{delay init } e \equiv \text{sync } c \text{ init } e$ in domain c , as an equivalence of the two relations. The kernel therefore has one temporal primitive — read a domain at its previous activation — and the single-domain semantics of the previous section is its diagonal. Under the always-active schedule, MEv coincides with Ev in every domain, so the earlier results are the one-domain special case rather than a replaced machine. A delay in a slow domain reads three global ticks back where a delay in a fast one reads one, with the same syntax.

A domain judgment Clocked $K c e$ rejects every other cross-domain reference: a reference stays in its domain or is agnostic, a delay needs a domain, and $\text{sync } c'$ switches the domain of its operand. Typing is unchanged and is blind to domains; the direct wire between two domains at the same value type is well typed and rejected only by the domain judgment. Placing the domain in the type instead was tried and set aside. Every pure mapping would need clock polymorphism, and nothing the type rejects is missed by the judgment.

9.3 Strictly before

A transport sees only source activations strictly before the destination tick. That is a choice with an observable alternative, and the alternative was built. A transport that lets simultaneously active domains see each other's current values makes the scheduler order observable: two priorities between the domains give two outputs. The strictly-before rule has no such parameter, and multi-domain evaluation is deterministic with no order between simultaneously active domains appearing in the semantics. It also makes cross-domain causality free. A transport's operand is never instantaneous, so Causal Δ is unchanged and no cross-domain cycle can be instantaneous. Every crossing costs one destination-visible step; "synchronous sub-domains evaluated in one instant" are, in this model, the same domain.

Every sync carries an explicit initial value, used at a destination activation with no earlier source activation. Totality extends to the multi-domain case — a causal, globally well formed design with well-typed inputs has a value in every domain at every tick — so the first activation is deterministic with the stated initial values. Semantic identity and dimension pass through transport untouched, by the typing rule; a crossing from Tilt@fast to Tilt@slow authorizes neither $\text{Tilt} \rightarrow \text{MotorAngle}$ nor $q \text{ Length} \rightarrow q \text{ Time}$.

9.4 Occurrences across domains

Optional values remain the right per-activation representation of an occurrence. What changes across domains is transport. A single-instant read loses events: under sync, two fast occurrences and one are indistinguishable at the slow tick, and a single occurrence followed by a quiet fast tick is dropped. The counterexample is against sync as an *event transport*, not against optional types.

The window model separates the two. What a destination should see is the source's occurrences at the source activations since the

destination's own previous activation. On tick sets, this window equals the source's accumulated log read at the current tick minus the log length read at the previous destination activation — two single-instant reads, a sync of a source-side accumulator and a delay of a cursor. Policies are functions of the window. `latest`, `count`, and `count with latest` each lose information and each identifies distinct windows; only the list is injective. So multiplicity and order are observable, buffering is required to keep them, buffering is a structured use of the existing state basis plus list data, and a bound on the buffer is a validation obligation.

Writing the buffer in the object language needs sequence data, and the kernel now has it: an ordinary list type $\text{list } \tau$, data exactly when τ is, with six registered operators (`nil`, `cons`, `length`, `take`, `reverse`, `head`) typed like every other primitive. No typing, evaluation, or domain rule was added, and every earlier theorem held unchanged. With it the buffer is five declarations: a source-side log `cons src (delay nil log)`, its transport `sync src nil log`, the length seen of the transported log, a cursor `delay 0 seen`, and the window `reverse (take (seen - cursor) logD)`. The correspondence is proved for every schedule, input, destination and tick: the window declaration evaluates, in the destination domain, to exactly the source's values at the window's activations, in order and with multiplicity. The list representation is injective on windows, whereas `latest`, `count`, a `fold`, and any summary bounded to a fixed number of newest entries each identify distinct windows — so a lossless summary is necessarily unbounded, and the list is the smallest general sequence representation tested. Capacity remains validation: for a finite horizon sufficiency is decidable and the least sufficient capacity is computed with a proof; for periodic schedules one destination period suffices at every horizon. Overflow is never implicit. Dropping the oldest or newest entries is the identity under sufficient capacity and changes the trace otherwise; the only policy that preserves the kernel semantics is to reject the deployment. An event type is still not needed: an event is a data-typed declaration in a domain, and its lossless cross-domain view is the window.

10 Physical Outputs Without Arbitration

10.1 The model

A declaration computes a value; it does not move hardware. Physical effect happens only through an explicit **drive edge** from a declaration to a **physical sink**:

- `OutputId` — the nominal identity of a sink, a logical actuator channel;
- $\Omega : \text{OutputId} \rightarrow \text{Option OutputSpec}$, a sink's accepted type and clock, declared by the deployment;
- $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ — the drive edges, a write-once per-declaration projection of the same shape as K ;
- $\text{DriveWF } \Omega K \Delta \beta$ — each edge is well formed when the driver's expected type *equals* the sink's accepted type and the driver's clock is the sink's clock;
- $\text{SingleDriver } \beta$ — at most one driver per sink: if βd_1 and βd_2 are both some o then $d_1 = d_2$;
- $\text{CompleteOutputs } \beta \text{ req}$ — every required sink is driven.

Nothing was added to types, typing, the domain judgment, the evaluation relation, or the grant. The binding neither coerces nor converts nor synchronizes. A declaration typed `Tilt`, or bare `q Angle`, cannot drive a `MotorAngle` sink. A sink that accepts a representation type needs an explicit `rep`-typed declaration in front of it, so the distinction between semantic target and hardware representation stays visible. A slow driver reading a fast value must sync it upstream, since a fast driver cannot drive a slow sink and the edge never synchronizes.

Sink identity is nominal for the same reason concept identity is. Keying the binding by type collides when two servos accept the same type; keying by concept conflates a concept with a device, since one concept may feed several; keying by declaration makes two declarations that both mean the steering motor into two sinks, and the multiple-driver counterexample cannot even be stated. “Desired steering angle” is a value; “the steering motor” is a resource; the kernel keeps them in different sorts.

10.2 One final driver

The principle is *many contributors, one explicit final driver*. Take two declarations driving one sink, each globally well typed, well clocked, causal, and individually well formed. Only SingleDriver fails, and it fails globally rather than at either edge. What has gone wrong is semantic: with two drivers the physical output is not a function of the tick, and there is a tick at which the sink receives two values. With one driver, the physical output is a partial function of the tick, and with SingleDriver it is unique wherever it exists:

$$\begin{aligned} \text{PhysicalOutput } S \Delta I \Omega \beta o t v &:= \exists d \text{ spec.} \\ \beta d &= \text{some } o \wedge \Omega o = \text{some spec} \\ \wedge \text{MEV } S \Delta I \text{ spec.clock } t \parallel (\text{declRef } d) v. \end{aligned}$$

Contributors are dependencies, not drivers. `base + corr -> final -> motor` passes every check; priority is an ordinary conditional in the single driver; blend, maximum, and clamp are ordinary declarations of the target type. Why arbitration must be explicit is best shown rather than argued: first-wins, last-wins, and maximum over the same value graph give three different physical outputs. A hidden policy is a design decision made on the designer’s behalf.

Binding an unbound declaration to an undriven sink is a refinement and preserves SingleDriver. Binding to an already-driven sink is invalid. Retargeting a sink’s accepted type, renaming a sink, or detaching an edge invalidates an unchanged design. Partial designs may leave sinks undriven; executable designs may not.

10.3 What was removed

The earlier draft of BDL had an effect row on the behavior judgment, action requests as values, and per-context policies that allowed, suppressed, transformed, and arbitrated requests, resolved in a dedicated phase of each tick. Each was tried in toy form against the single-driver model, and the outcome is narrow. Direct effect rows — the set of sinks a declaration drives — are exactly the drive edges, and single-driver is exactly their pairwise disjointness. Propagated rows, which also include the sinks of everything a declaration reads, flag a valid design in which a display reads the driver. Action values move the conflict into the collector that consumes them, which must then be a policy, which is the single driver by another name. None of this bears on richer effect systems; it says that these formulations add no rejection the single-driver rule lacks. Per-context action policies no longer exist as a mechanism, and the StateHandler cases that involved them — event-latched activation with exit-wins, state-local output choice, nested choice with an output — are ordinary declarations with one driver and were run as such.

11 Target-Specific Hardware Validation

Everything to this point is board-independent. A design that is typed, causal, clock-consistent, and output-complete may still not fit the microcontroller it is to run on, and that question is answered by a validation layer that never touches the kernel.

11.1 Resources, capabilities, requirements

A target is a finite table. A **resource** — a pin — has an identity, a list of **capabilities** from a shared vocabulary (digital in/out, PWM, analog in, interrupt, the I2C, SPI, and UART lines), and, per capability, the **unit** backing it, such as the timer behind a PWM pin or the peripheral behind a bus line. A **hardware description** is a list of resources with

a per-capability sharing policy: bus lines are shareable, everything else is exclusive.

A design’s needs are **requirements**. Each has an identity, one capability, an optional fixed resource for a manual pin choice, and an optional membership in a group with a unit relation, *same* or *distinct*:

$$\begin{aligned} \text{Requirement} = \langle & id, cap, \\ & fixed : \text{Option ResourceId}, \\ & group : \text{Option } (\mathbb{N} \times \text{UnitRel}) \rangle. \end{aligned}$$

Requirements are generated from the physical sinks of the previous section by a **device kind**: an H-bridge channel needs a PWM line and a digital output; an I2C sensor needs SDA and SCL on the same unit; a quadrature encoder needs two interrupt lines. The pipeline is

OutputId $\rightarrow$ DeviceKind $\rightarrow$ Requirements $\rightarrow$ solve $\rightarrow$ Assignment,

and the sink never enumerates pins, so swapping the board is re-solving the same requirements with the design untouched. Requirement identity is independent of sink and declaration identity: one sink generates several requirements, and two identical PWM needs must be two distinct variables.

An **assignment** is a list of requirement–resource pairs. It is **partially valid** when every entry is supported — the resource has the capability and any fixed choice is respected — and every pair is **compatible**: two entries on the same resource must have the same capability and it must be shareable, and two entries in the same group must have equal units under *same* and different units under *distinct*. It is **valid for** a requirement list when partially valid and covering exactly that list; the target is **satisfiable** when such an assignment exists.

11.2 A sound and complete solver

Every constraint is unary or binary, so validity is prefix-closed, and an exhaustive depth-first search that prunes on unary support and pairwise compatibility with the current prefix is complete as well as sound. Both are proved: if the solver returns an assignment it is valid for the requirements, and if a valid assignment exists the solver returns one. Feasibility of a finite instance is therefore decidable, and the solver runs inside the proof checker by decide. The instance is small enough that a general constraint solver [8] is unnecessary; the claim is about this scope, not about embedded allocation in general.

Extension of a target — adding capabilities, units, or sharing to existing resources — preserves every valid assignment. Removing a resource, adding or strengthening a requirement, or fixing a pin may not.

11.3 Case study: an Arduino Nano

The Arduino Nano’s digital and analog pins are encoded as a table: PWM on D3, D5, D6, D9, D10, and D11 backed by timers 2, 0, 0, 1, 1, and 2; external interrupts on D2 and D3; I2C on A4 and A5. A design with four H-bridge motor channels and one I2C inertial sensor is satisfiable, and the solver’s output is the assignment a tool would present:

```
M1 -> D3 / D0      M2 -> D5 / D1
M3 -> D6 / D2      M4 -> D9 / D4
IMU -> A4 / A5
```

Two counterexamples fix the boundary between kernel and validation. Seven independent PWM actuators pass every kernel condition — globally well formed, well clocked, causal, every drive edge well formed, single-driver, output-complete — and are unsatisfiable on the Nano, which has six PWM pins. The same requirements are satisfiable on a larger mock board with six more PWM pins on three more timers. Two interrupt lines and six PWM lines meet every capability *count* exactly, two and six, and are unsatisfiable, because D3 is both

the only second interrupt pin and one of the six PWM pins. Capability counting is not feasibility. Further examples establish that two PWM requirements pinned to the same pin are rejected, that two I2C sensors on A4/A5 are accepted since allocation is not all-different, that a manual pin choice can turn a satisfiable design unsatisfiable while a consistent one is honoured, that four PWM lines required on independent timers are unsatisfiable on the Nano’s three timers though six PWM pins exist, and that TX and RX pinned to one UART unit stay together.

The explanation facility reports the first dead end under greedy placement. For the seven-actuator design it names the seventh actuator and, for each PWM pin, the actuator blocking it. This is a conflict under one placement order, not a minimal unsatisfiable core, and it is meaningful only when the solver has already returned no assignment.

11.4 Two kinds of evidence

Hardware feasibility is evidence about the pair (design, target), and it is not monotone in the sense the preservation theorems require. Six actuators are satisfiable; adding a seventh — a monotone extension of the design by a declaration and its sink — is not. The distinction drawn earlier between evidence that survives refinement and evidence that is rechecked after every change is here concrete. Commitments discharged compositionally survive EnvRefines; deployability on a target is re-solved; and the two are never merged. This is why the workspace reports them as different states.

What the layer does not model should be stated as plainly. Voltage, current, thermal budgets, memory, processor load, deadlines, bus bandwidth, torque, travel, and PWM frequency values are outside it. Some of these need summation constraints that are not binary, and while the architecture leaves room for a compatibility predicate over sets, nothing here establishes it. Units are one integer per capability per resource, which models which timer or which UART but not timer modes.

12 Elaboration and Tool Architecture

The surface editor and kernel are connected by an elaboration function from surface designs to kernel environments with diagnostics. No elaborator has been implemented; this section states the passes that the kernel’s structure implies, and what each is responsible for.

Name and signature resolution. Resolve semantic properties to identities, Mapping signatures to declaration interfaces, contexts, device kinds, units, and imported components. At this stage an unresolved `Tilt` $\rightarrow$ `Brightness` mapping already has a stable identity and an expected type, and every reference to it is typed.

Formula elaboration. For each Mapping with a definition, check the definition against the declared codomain under the grant of the declaration’s own signature. A scalar formula attached to a `Brightness` output elaborates to `mkBrightness(...)` because the surrounding signature announces the concept; the constructor is inserted by the elaborator and is legal only there. Units elaborate to scaled dimensioned literals. Curve, example, and component definitions elaborate to the same realization form.

Temporal lowering. Surface temporal phrases lower to the declaration shapes of the derived-operator table. A context lowers to an activation declaration, an entry declaration, gated and reset local state, and a conditional in each output’s driver. A reusable stateful component is instantiated into fresh declarations at each use, because temporal state cannot occur under a binder.

Domain assignment. Domains are declared, so this pass records the clock environment and checks the domain judgment. A reference across domains without a transport is reported at the reference, in the designer’s names for the two domains, with the two legitimate

resolutions — transport the value with a stated initial value, or move the reader to the source’s domain — stated in terms of their behavioral consequence.

Output binding. Record the drive edges and check type and clock equality, single-driver, and, for executable designs, completeness. Where several contexts would drive one output, the elaborator emits one driver whose body selects among them; the selection is visible in the elaborated design.

Hardware validation. Generate requirements from device kinds, run the solver against the selected target, and attach the assignment or the explanation to the bindings.

Normalization and erasure. After checking is complete, semantic identities, dimensions, and domains carry no computational content and may be erased; erasure is proved sound for identities and for dimensions. Nominal wrappers introduced by elaboration cancel, $\text{rep}(\text{mk}_s e) \rightsquigarrow e$, and the flattened program is checked under the universal grant because every construction was authorized at its declaration. Unfolding all realizations into a closed term preserves typing on the delay-free fragment and agrees with tick-by-tick evaluation on first-order designs; the higher-order case holds up to closure equivalence and was not formalized. Erasure is applied selectively at boundaries the checker cannot see through — supplied blocks, device bindings, the public interface of generated code — where wrappers are retained so that the host compiler continues to check what BDL cannot.

The kernel never depends on normalization to decide type equality; it is not dependently typed, and normalization is an analysis and code-generation instrument. Floating-point arithmetic is not associative, so any symbolic normalization over the reals must record the resulting numerical deviation as an obligation rather than silently altering the property being checked.

13 Mechanized Design-Space Evaluation

The kernel of this paper was obtained by a method, and the method is part of the contribution. Each phase of the development took a family of candidate constructs from the earlier draft, formalized the smallest plausible version and its alternatives in Lean 4 without external libraries, and attacked each with the same questions. What does it reject that the others accept? What does it accept that it should not? Is it a special case of another? Which operations on a design are refinements under it, and which are edits? Constructs were promoted from the experiment modules to the kernel only after surviving, and the theorems about them were re-proved in the promoted form.

13.1 Claim strength

Formal counterexamples reject the specific tested design, not every conceivable design in the same informal family. Every conclusion in the development is therefore labelled with one of a fixed set of strengths, and this paper has used the same vocabulary throughout: **formally proved** (a theorem in the development); **formally rejected by counterexample** (a mechanized example breaks the specific formulation); **reduction by proof** (a theorem shows one design is a special case or erasure of another); **tested formulation redundant** (the one formulation examined duplicates an existing mechanism; the family is not excluded); **engineering preference** (argued, not proved); and **not ruled out**. Nothing in the development is a proven impossibility, and “minimal” in this paper always means minimal among the tested designs.

13.2 What the kernel contains and why

Construct	Status	Basis
declaration with frozen type, monotone commitments, write-once realization	kernel	preservation theorems; each edit witnessed

Construct	Status	Basis
typing through the type view only	kernel	sufficient by theorem; necessary by counterexample
Evidence.Monotone as a condition on validation	imposed by kernel	preservation fails without it
nominal sem SemanticId	kernel	baseline accepts the invalid wire; metadata checker η -evaded; concept-as-declaration has category errors
representation binding Θ ; rep free; mk under Grant.of signature	kernel	unrestricted mk/rep bypasses identity; observation-only cannot realize a mapping
dimensions q Dim, algebra in Prim.ty	kernel	numeric baseline is the erasure; no dimension rule needed
units	surface	scaled literal; value changes, type does not
delay init e, data-typed, top-level; tick semantics	kernel	deterministic; total on causal designs; restrictions forced by totality
Signal τ in Ty	removed	inhabited by exactly the terms of τ
Event τ	removed (single domain)	opt τ streams are the multiplicity- ≤ 1 streams
temporal operators	surface	each executed as a self-delayed declaration
Causal on instantaneous dependency	kernel	replaces structural acyclicity; conservative for lambda-guarded cycles
ClockId, clock environment, Clocked	kernel	direct wire ambiguous without it; equal rate is not same domain
clock in the type	removed	forces polymorphism on every pure mapping
sync src init e	kernel	one transport; delay is its diagonal; same-tick visibility makes scheduler order observable
list τ with six operators	kernel	a lossless window is unbounded sequence data; no new typing, evaluation, or domain rule
buffer primitive, Event τ across domains	removed	buffer = five declarations over delay/sync/lists; proved equal to the window
event policies, capacity	surface; validation	policies are computations over the window; only reject-deployment preserves semantics
products $A \times B$, list recursor fold, eq on every data type	kernel	function encodings are not data and cannot be state; one eliminator derives every collection operation; the kernel's only capability evidence is Data
structural order on data (mode < mode, None < Some, lexicographic pairs)	removed	no behaviour-design meaning; lt compares quantities, an ordered concept compares by declaration through its representation
rank-1 generic definitions	surface	families of monomorphic terms instantiated by matching; no type variable, $\forall$ or λ in the kernel
sets, intervals, records, finite quantifiers, min/clamp/any/all/map	surface	definitions over pairs, lists and fold; $x \in \{...\}$ is list membership; forall x in xs is all
higher-rank types, type-classes, existentials, row polymorphism	removed	every candidate use is rank ≥ 2 with a rank-1 replacement; hiding is component instantiation; the constraint vocabulary is the closed {Data, Eq, Ord}
OutputId, drive edge, DriveWF, SingleDriver	kernel	two drivers make the output non-functional; hidden policy observable
effect rows, action values, arbitration	removed	rows duplicate edges or false-positive; values relocate the conflict
hardware resources, requirements, solver	validation	feasibility is target-relative and not monotone

The tested StateHandler behaviors, the event buffer, and the lambda-guarded cycle are the recorded boundaries of these results.

13.3 Scale and trust base

The development builds with Lean 4.33.1 with no sorry. The axioms used are propositional extensionality and quotient soundness, the latter only through function extensionality, and no classical choice. The kernel and validation layer occupy eleven modules, with eight experiment modules holding the alternatives and counterexamples; every trace, assignment, and unsatisfiability result reported here was obtained by running a proved-sound interpreter or solver inside the proof checker. Several theorems are recorded as trivial by definition

— the typing half of client stability is a one-liner, and the well-formedness of a refinement target is unused because the invariant was moved into the definition — and they are reported as such rather than presented as content.

14 Evaluation Plan

No user study has been run, and no elaborator, editor, or firmware generator exists. This section states what should be measured and how, without reporting results.

14.1 Expected cognitive advantages

Lower viscosity. Changing a transfer function edits one Mapping definition rather than a procedural chain of read/compute/store/write nodes.

Reduced hidden dependencies. Semantic signatures expose what a relationship consumes and produces, while the kernel rejects incompatible connections before device code exists.

Reduced premature commitment. An unresolved declaration allows a designer to commit to a relationship without committing to its implementation, sensor, or exact parameters.

This last item requires a qualification that the rest do not. Signature-first authoring is not claimed to be the spontaneous habit of every designer. One of the authors, working on a hardware project, adopted it when the complexity of a subsystem exceeded what could be held in view at once, and did not adopt it on simpler tasks, where a signature and its definition were written in a single motion. The hypothesis is therefore narrower than a claim about how designers think. It is that conventional tools provide no legal position for an undefined relationship, so the strategy cannot be adopted even when it would help, and that BDL supplies that position at no cost. Whether inexperienced designers take up the strategy once it is available, and whether doing so improves their designs, is an empirical question and is included in the study below.

Better role expressiveness. Contexts, Mappings, and temporal modifiers correspond to product-design concepts rather than generic program-control constructs.

Improved error locality. A mismatch is attached to a product relationship or binding rather than surfacing later as an embedded runtime fault, and a hardware infeasibility is attached to the binding that causes it.

14.2 Comparative study

A first controlled study should compare BDL against at least two baselines: a statechart-based prototyping environment and a node-based or Arduino-style implementation workflow. Participants should be industrial-design students and practitioners with limited professional software-engineering experience.

Tasks should include specifying a sensor-to-actuator mapping; adding temporal qualification such as “for 300 ms”; adding an orthogonal safety override that competes with an interaction context for one output; replacing a sensor with a different sample rate; relating a slowly updated quantity to a fast interaction, which forces a cross-domain decision; choosing a board that cannot accommodate the design; and modifying a mapping late in the task.

Primary outcomes should not be limited to task time or a usability scale. More important measures are semantic errors in the final behavior; the number of implementation-only concepts participants must manipulate; time to detect an impossible or conflicting behavior; fidelity between verbal design intent and the elaborated model; the number and diversity of behavior alternatives explored; the quality of handoff to an engineer who did not observe the authoring session; subjective confidence calibrated against actual correctness;

and whether, and at what level of task complexity, participants declare a relationship before defining it when the tool permits both.

The last measure tests the hypothesis stated above rather than assuming it. Because signature-first authoring may be a strategy that appears only above a complexity threshold, the tasks should vary in scale, and the default state of a newly created Mapping Block is itself a manipulable factor: a block that opens onto an empty formula editor and one that opens onto a signature with an explicitly legal undefined body invite different first actions. A small comparison of these two defaults is considerably cheaper than the full study.

The interaction model of this paper adds hypotheses of its own. Whether the single-driver diagnostic leads participants to an explicit combination block they can later read, whether the cross-domain question is answered correctly for a safety condition, and whether participants distinguish a valid design from a deployable one when the workspace reports them separately, are each measurable in the tasks above.

14.3 Field study

A controlled study cannot establish whether the representation fits real design practice. A second phase should embed the tool in a semester-long product-design studio or an industry project. The study should observe where unresolved declarations persist, which semantic types designers invent, where they request escape hatches, how often the single-driver condition is met by a combination rule the designer finds natural, and how often engineers reinterpret or replace BDL artifacts during implementation. This field evidence is necessary before claiming that the language is native to industrial design rather than merely pleasant to its authors.

15 Related Work

15.1 Physical prototyping and design tools

Phidgets reduced the implementation cost of physical interaction by presenting hardware components through a uniform software abstraction [1]. d.tools integrated physical prototyping, statechart-based behavior, testing, and analysis for designers [2]. Exemplar addressed the same authoring cost from the opposite direction, letting designers demonstrate sensor behavior and having the system infer the recognizer [9]; in BDL that technique is one of several ways to supply a definition, attached to a signature that already fixes the relationship's semantic boundary. The difference from these systems is not that they could not represent behavior. It is that their dominant representation still asks designers to formulate substantial portions of behavior in an operational structure; BDL investigates whether the typed semantic relationship can be the primary artifact, with operational machinery elaborated underneath.

15.2 Dataflow and model-based engineering tools

LabVIEW established the graphical dataflow instrument-control paradigm; Simulink with Stateflow combines dataflow blocks with hierarchical state machines; Modelica models physical systems through acausal equations with units and dimensions [10], [11], [12]. These systems are considerably more capable than what is proposed here. The distinction is in the primary artifact and the intended author. In each of them the artifact is an executable model whose blocks denote computation and the author holds an engineering model; BDL's artifact is a set of typed relationships that need not yet compute anything, and the author holds a product model. SysML and model-based systems engineering address precise system structure, requirements, and verification at a broader level [13]; BDL is intended as a front end for early design that could later export into such representations, not as a replacement.

15.3 Synchronous languages and functional reactive programming

The kernel's temporal basis is that of the synchronous dataflow tradition. Lustre's pre with an initial value, in a declaration-per-stream setting, is the delay primitive here [6]; the rule that a value computed at an instant is visible at the next instant, applied across domains, is the strictly-before rule; and causality as a static property follows the same line [14]. Esterel established the synchronous hypothesis under which these semantics are deterministic [15]. Where these languages recover clocks by a clock calculus [7], BDL requires domain identity to be declared, for the reason given in the clock-domain section. Zélus extends the lineage to hybrid systems [16], which is the direction in which the continuous-dynamics limitation noted below would have to be addressed.

Functional reactive programming established behaviors and events as compositional abstractions for time-varying computation [17], and FrTime gave a dynamic dataflow embedding with formal semantics [18]. BDL's reactive core is much more restricted, and the restriction is a result rather than a starting point: under a tick semantics with one domain, a signal type rejects nothing and an event type is an optional-valued stream, so neither appears in the kernel.

15.4 Typed holes and structure editing

Hazelnut demonstrated that incomplete structured terms can remain statically meaningful under bidirectional typing [19], and Hazel extended this to live evaluation around holes [20]. BDL takes the idea that incompleteness is a first-class static state and relocates it. The kernel object is a named declaration whose realization is optional; holes are positional in the Hazelnut tradition and named here; and the stability result concerns clients of a declaration under refinement rather than the typing of the incomplete term itself. This relocation reduces to ordinary interface/implementation separation and is not a new hole calculus.

15.5 Dimensional typing

Dimensional typing follows the units-of-measure line begun by Kennedy [21]. The contribution here is only the placement — the algebra in primitive operator types with no dimension-specific rule, and the separation of dimension from nominal identity — together with the observation, mechanized, that the numeric baseline is the erasure.

15.6 Effects

An earlier draft of BDL borrowed the separation between operation and interpretation from algebraic effects [22] and anticipated scoped effects for context-sensitive interpretation [23]. The kernel presented here has no effect system. The negative result is stated narrowly in the section on physical outputs and does not bear on effect systems in general; it bears on the formulations tried for this design problem, where a single explicit driver per output expressed everything the request-and-policy model expressed and made visible what it hid.

15.7 Resource allocation

The hardware validation layer is a finite constraint satisfaction problem with unary and binary constraints [8], and its solver is a plain backtracking search whose soundness and completeness are proved. No claim is made relative to the embedded co-design literature; the layer's contribution is architectural — the design is never an input to the solver, and feasibility is kept as a separate, non-monotone kind of evidence — rather than algorithmic.

16 Discussion and Limitations

16.1 Why the signature is a design object

The most consequential decision in BDL is treating a declared relationship as visible product intent. In programming, a signature is often documentation and a static contract around code. In BDL it

can precede any code-like definition and remain useful on its own: $?f : \text{Tilt} \rightarrow \text{Brightness}$ says that the designer has committed to a causal design relationship and to its semantic boundary, and does not say how the mapping is computed. Clients are typed against the type view and depend on the interface only through it and through monotone evidence, so the partial commitment is not a weaker form of a complete one. It is the form on which everything downstream already rests.

16.2 Nominal identity in three places

BDL makes the same choice three times. On the axis of quantity, `sem` makes semantic identity nominal: two concepts of equal representation are distinct, and moving between them is a declared relationship. On the axis of time, `ClockId` makes temporal identity nominal: two domains of equal rate are distinct, and moving between them is a sync with an initial value. On the axis of effect, `OutputId` makes sink identity nominal: two sinks of equal accepted type are distinct, and driving one is an explicit edge. In each case the identity is independent of representation, of rate, and of type respectively; in each case the crossing is a visible artifact rather than a compiler action; and in each case the alternative — identity by representation, by rate, or by type — collided or was ambiguous for a mechanized reason. The three are not all placed alike. Semantic identity is in the type, while clock and sink identity are projections beside the interface checked by separate global judgments, because putting the clock in the type forces polymorphism on every pure mapping. Symmetry was not a design goal; it is what remained.

16.3 Formal limits

The calculus assumes discrete logical clocks, deterministic primitives, and finite temporal state. It does not define continuous dynamics, probabilistic sensor estimates, distributed clock uncertainty, or hybrid semantics. These are substantial extensions, not footnotes.

Several boundaries are internal to what was formalized. Causality is conservative for lambda-guarded cycles. The buffer correspondence assumes an input source. The equation language is total first-order-data computation with higher-order functions and one list recursor; sum types are encoded as a tag with an optional payload rather than added, and the library's evaluation lemmas assume the predicate argument implements a Boolean function. The agreement between unfolding and tick-by-tick evaluation is proved for first-order designs only. The `StateHandler` reduction covers the tested cases and not contexts with their own clocks. Hardware validation covers discrete pin and peripheral allocation with unary and binary constraints, and no numeric electrical or timing property. The explanation facility reports a first dead end, not a minimal core. Interface-level references — commitments that mention other declarations — are not modelled, so the dependency graph is over realizations only. Whether a realization may delegate its grant to a higher-order argument is untested. Affine units are modelled for conversion and display only; the point/difference sort their arithmetic needs is not.

16.4 Risks of the design

Semantic-type proliferation. If every semantic distinction creates a visible type, the editor may become bureaucratic. The system needs reusable type libraries and sensible defaults, and the study should record which types designers invent.

Formula anxiety. Not every designer wants to write equations. Formula authoring must coexist with curves, examples, and direct manipulation.

Hidden elaboration. A context that elaborates to an activation declaration, an entry declaration, gated state, and a conditional driver is, in the elaborated design, several objects the designer did not draw. Hiding this can make runtime behavior mysterious. The explanation view of the interaction model is the proposed answer;

whether designers use it, and whether it explains what they came to ask, is untested.

False confidence. A formally typed diagram can look verified even when no physical property has been checked, and a typed, causal, clock-consistent, output-complete design can be unplaceable on the chosen board. The workspace states of the interaction model exist to keep these apart, and the risk is sharpest for obligations discharged by declaration rather than by analysis.

Complexity migration into tooling. Much of what was removed from the kernel — event policies, output selection, context semantics — reappears as elaboration. The kernel is smaller and better understood; the elaborator is larger and does not yet exist. The claim that the surface is “only syntax” over the kernel is a claim about tested cases, and the untested cases are the ones most likely to demand a kernel extension.

Usability hypotheses unestablished. Every statement in this paper about what designers find natural is a hypothesis, including every sentence of the interaction model. The one anecdote reported is a single author's practice on a single project.

17 Conclusion

Modern industrial products increasingly combine physical form with sensing, computation, and control, yet designers still lack a behavior medium with the immediacy that CAD provides for geometry. BDL proposes that the missing medium should not be a friendlier version of procedural programming. It should be a language in which typed product relationships are first-class design artifacts, and in which an unresolved relationship is a legal state of the design rather than a defect in a program.

What that looks like in use is a workspace in which a designer names what the product is about, draws the relationships between those things, and leaves each undefined until there is something to say; refines them locally, by formula or curve or example, without the diagram changing shape; states time as a qualifier rather than as a timer; gives situations a name and a boundary rather than a transition table; is asked for one final target where two behaviors reach for one output, and for one stated way of seeing across a boundary where two quantities move at different speeds; and chooses a board last, receiving either a pin allocation or a conflict, with the design itself untouched either way.

The kernel that supports this is small, and it is small for reasons that were checked. A declaration has a frozen type, growable public commitments, and a write-once body; clients are typed against the type view, and refinement preserves what they established while edits reopen it. Semantic concepts are nominal, represented through a write-once binding, and constructed only where a signature announces them; dimensions are carried by the types of primitive operators. One temporal primitive reads a clock domain at its previous activation, and single-domain delay is its diagonal; evaluation is deterministic and total exactly on causal designs; every designer-facing temporal operator is a shape over it. Clock domains are nominal and checked by a judgment rather than a type; crossings are explicit, initialized, and strictly earlier. Physical outputs are nominal sinks with one driver each, and every combination of behaviors is ordinary computation upstream of the drive edge. A separate validation layer decides whether the design fits a board, and its evidence is kept apart from the evidence that survives refinement.

Each of these is minimal among the designs that were tested, and the paper has tried to say, for each, what was proved, what was rejected by counterexample, and what was preferred. What has not been built — the elaborator, the editor, the firmware path, and the studies — is where the claims about designers would be tested. The research question is not whether designers can be taught a simpler

programming language. It is whether product behavior can become a *design material* whose structure is intuitive at the surface and rigorous underneath, and the kernel presented here is the part of that question that can now be stated precisely.

References

- [1] S. Greenberg and C. Fitchett, “Phidgets: Easy Development of Physical Interfaces through Physical Widgets,” in *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2001, pp. 209–218. doi: 10.1145/502348.502388.
- [2] B. Hartmann *et al.*, “Reflective Physical Prototyping through Integrated Design, Test, and Analysis,” in *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2006, pp. 299–308. doi: 10.1145/1166253.1166300.
- [3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, in Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [4] T. R. G. Green and M. Petre, “Usability Analysis of Visual Programming Environments: A ‘Cognitive Dimensions’ Framework,” *Journal of Visual Languages & Computing*, vol. 7, no. 2, pp. 131–174, 1996, doi: 10.1006/jvlc.1996.0009.
- [5] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: 10.1016/0167-6423(87)90035-9.
- [6] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The Synchronous Data Flow Programming Language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991, doi: 10.1109/5.97300.
- [7] J.-L. Colaço and M. Pouzet, “Clocks as First Class Abstract Types,” in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [8] R. Dechter, *Constraint Processing*. Morgan Kaufmann, 2003.
- [9] B. Hartmann, L. Abdulla, M. Mittal, and S. R. Klemmer, “Authoring Sensor-Based Interactions by Demonstration with Direct Manipulation and Pattern Recognition,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2007, pp. 145–154. doi: 10.1145/1240624.1240646.
- [10] National Instruments, “LabVIEW.” 2024.
- [11] The MathWorks, “Simulink and Stateflow.” 2024.
- [12] Modelica Association, “Modelica Language Specification, Version 3.6.” 2023.
- [13] Object Management Group, “OMG Systems Modeling Language (SysML) Version 2.0.” [Online]. Available: <https://www.omg.org/spec/SysML/2.0>
- [14] J.-L. Colaço, B. Pagano, and M. Pouzet, “A Conservative Extension of Synchronous Data-flow with State Machines,” in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: 10.1145/1086228.1086261.
- [15] G. Berry and G. Gonthier, “The Esterel Synchronous Programming Language: Design, Semantics, Implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992, doi: 10.1016/0167-6423(92)90005-V.
- [16] T. Bourke and M. Pouzet, “Zélus: A Synchronous Language with ODEs,” in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: 10.1145/2461328.2461348.
- [17] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [18] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.
- [19] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: A Bidirectionally Typed Structure Editor Calculus,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: 10.1145/3009837.3009900.
- [20] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, “Live Functional Programming with Typed Holes,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, p. 14:1–14:32, 2019, doi: 10.1145/3290327.
- [21] A. Kennedy, “Relational Parametricity and Units of Measure,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: 10.1145/263699.263761.
- [22] G. D. Plotkin and M. Pretnar, “Handling Algebraic Effects,” *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: 10.2168/LMCS-9(4:23)2013.
- [23] Z. Yang, M. Paviotti, N. Wu, B. van den Berg, and T. Schrijvers, “Structured Handling of Scoped Effects,” *Proceedings of the ACM on Programming Languages*, vol. 6, no. ICFP, 2022, doi: 10.1145/3547654.